

Task 1

1. A sender can encrypt a command with AES-CTR, which will turn a key into a stream of random bytes and XORs them with whatever message is being tried to send. The receiver does the same XOR to get the message back. There is zero lock on the message so the receiver has no way to see if the message has been changed or used. In this task, the relay is an attacker who can see and change traffic but doesn't have the key. It sits in the middle.
2. In my experiment, two big things occurred. First, the attacker knew the message format so it knew READ sits at a particular spot. It XORs READ and WIPE into the ciphertext and it never needed a key. The receiver decrypted the message without a error and saw WIPE instead of READ. The output showed $C \text{ XOR } C' \text{ equal to } P \text{ XOR } P'$, which means that whatever the attacker XORs into the ciphertext shows up as the same change in the plain message. Second, the attacker sent a valid ciphertext twice, and the receiver decrypted and ran it both times, because nothing told it the message was a repeat.
3. The root cause is that encryption can only HIDE a message. It cannot guarantee that a message can be modified in transient or even fresh.

```

ubuntu@ubuntu2404: ~/csce465-agentsec/hw2
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ nano baseline_ctr.py
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ source ../venv/bin/activate
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ python baseline_ctr.py
[1] Original
    plaintext : b'{"action": "READ", "path": "notes.txt"}'
    ciphertext: 80d3187696a84d404b6f2b7b97618e746fd1e947435ed9bdf49f0ad08bceeb7743474134

[2] Relay modifies ciphertext (key never used)
    forged ct : 80d3187696a84d404b6f2b7e9b708f746fd1e947435ed9bdf49f0ad08bceeb7743474134

[3] XOR relation  $C \oplus C' = P \oplus P'$ 
    C xor C' : 00000000000000000000000050c11010000000000000000000000000000000000000
    P xor P' : 00000000000000000000000050c11010000000000000000000000000000000000000
    equal?    : True
    READ xor WIPE = 050c1101

[4] Receiver accepts forged ciphertext with no error
    receiver executed: {'action': 'WIPE', 'path': 'notes.txt'}

[5] Replay: same original ciphertext delivered twice
    receiver executed: {'action': 'READ', 'path': 'notes.txt'}
    receiver executed: {'action': 'READ', 'path': 'notes.txt'}
    READ was processed 2 times from one sender transmission
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ █

```

Task 2

Diffie-Hellman key establishment

- The gateway and node both pick a new private value and only send the public value. Both compute the same shared secret Z and an eavesdropper who can only access public values cannot compute it. Z is then run through the task's KDF to get session keys. On its own DH gives secrecy against a passive attacker but doesn't help about who you are talking to.

Digital signatures and Entity auth

- Both sides sign role || SHA-256(transcript) with long-term RSA-PSS keys and a peer checks that signature against the already trusted public key. A valid signature shows that the sender has the private key which is entity auth. Because a signature covers a session, it proves the peer is taking part in this exchange now, and not just that it at one point held the key. RSA signs here only and never encrypts which means it authenticates the exchange without touching any app data.

Nonces and Replay resistance

- Both sides contribute a fresh 16-byte random nonce (which is a part of the signed transcript). This makes the messaging here unique, so if an attacker replays an old handshake, the transcript does not match the new nonces and fresh DH values so it falls through. Even a successfully replayed old message wouldn't give anything without a matching DH private value.

Forward secrecy

- DH values are generated per session and destroyed once keys are changed. If a long term RSA key is stolen later, the attacker can pretend to be that in future handshakes. The attacker cannot recover past session keys since they depend on destroyed values.

Key separation

- The KDF uses distinct labels to produce separate keys for a few reasons. A gateway to node encryption, a gateway to node MAC, a node to gateway encryption, a node to gateway MAC, and the ID. They are all bound to a transcript hash. One issue wouldn't help against another key, and a message sent in one direction cannot be passed off as one sent in another.

```
ubuntu@ubuntu2404: ~/csce465-agentsec/hw2
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ python handshake.py
handshake ok, keys match: True
session_id: 407f3a4a0adfb736
TH: 7c5d1f2368cd8094319b5446b5194051c34214c85a9108e1db91a65df836b613
modified nonce: rejected -> invalid peer signature
modified DH public value: rejected -> invalid peer signature
unexpected peer identity: rejected -> unexpected peer identity
reflected hello: rejected -> unexpected peer identity
reflected hello, identity rewritten: rejected -> reflected handshake message
signature by wrong RSA key: rejected -> invalid peer signature
wrong peer RSA public key: rejected -> invalid peer signature
malformed transcript length: rejected -> malformed transcript: declared length exceeds data
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$
```

Task 3

```
(.venv) ubuntu@ubuntu2404: ~/csce465-agentsec/hw2$ python secure_record.py  
gateway -> node: 010000000000000000000100000024c5fc76834f1f608af4e958dc8204198c86b57603327e77290210cc5dbdd75f5663975d630096f8122b929c1c90a37ec0a232aa8d7f8e2a971afd00177561398d325b09eb  
node opened: (1, b'{"action": "READ", "path": "notes.txt"}')  
node -> gateway opened: (2, b'{"status": "ok"}')  
modified ciphertext: rejected -> AuthenticationFailure: MAC verification failed  
modified message_type: rejected -> AuthenticationFailure: MAC verification failed  
modified sequence: rejected -> AuthenticationFailure: MAC verification failed  
modified length: rejected -> MalformedRecord: declared ciphertext length does not match record  
modified tag: rejected -> AuthenticationFailure: MAC verification failed  
untouched record still accepted: (1, b'{"action": "READ", "path": "notes.txt"}')  
replayed record: rejected -> ReplayError: unexpected sequence number (reply or reordering)  
own record reflected to sender: rejected -> WrongDirection: record is for the other direction  
reflected, direction byte rewritten: rejected -> AuthenticationFailure: MAC verification failed  
gateway accepts it normally: (2, b'hello gateway')  
out-of-order record: rejected -> ReplayError: unexpected sequence number (reply or reordering)  
in order: (1, b'first') (1, b'second')  
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$
```

Task 4

```
ubuntu@ubuntu2404: ~/csce465-agentsec/hw2
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ python -m pytest tests -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/ubuntu/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/ubuntu/csce465-agentsec/hw2
collected 29 items

tests/test_adversarial.py::test_valid_handshake_and_bidirectional_messages PASSED [ 3%]
tests/test_adversarial.py::test_sequence_numbers_start_at_zero_and_increment PASSED [ 6%]
tests/test_adversarial.py::test_modified_ciphertext_rejected_and_state_unchanged PASSED [ 10%]
tests/test_adversarial.py::test_modified_tag_rejected PASSED [ 13%]
tests/test_adversarial.py::test_modified_authenticated_header_rejected[0-MalformedRecord] PASSED [ 17%]
tests/test_adversarial.py::test_modified_authenticated_header_rejected[1-WrongDirection] PASSED [ 20%]
tests/test_adversarial.py::test_modified_authenticated_header_rejected[2-AuthenticationFailure] PASSED [ 24%]
tests/test_adversarial.py::test_modified_authenticated_header_rejected[9-AuthenticationFailure] PASSED [ 27%]
tests/test_adversarial.py::test_modified_authenticated_header_rejected[10-AuthenticationFailure] PASSED [ 31%]
tests/test_adversarial.py::test_modified_authenticated_header_rejected[14-MalformedRecord] PASSED [ 34%]
tests/test_adversarial.py::test_truncated_record_rejected PASSED [ 37%]
tests/test_adversarial.py::test_replayed_record_rejected PASSED [ 41%]
tests/test_adversarial.py::test_out_of_order_record_rejected PASSED [ 44%]
tests/test_adversarial.py::test_record_reflected_to_sender_rejected PASSED [ 48%]
tests/test_adversarial.py::test_reflected_record_with_rewritten_direction_fails_mac PASSED [ 51%]
tests/test_adversarial.py::test_wrong_session_keys_rejected PASSED [ 55%]
tests/test_adversarial.py::test_sender_never_reuses_iv PASSED [ 58%]
tests/test_adversarial.py::test_sequence_exhaustion_refuses_to_seal PASSED [ 62%]
tests/test_adversarial.py::test_wrong_rsa_public_key_rejected PASSED [ 65%]
tests/test_adversarial.py::test_invalid_signature_rejected PASSED [ 68%]
tests/test_adversarial.py::test_signature_by_untrusted_key_rejected PASSED [ 72%]
tests/test_adversarial.py::test_reflected_handshake_message_rejected PASSED [ 75%]
tests/test_adversarial.py::test_modified_nonce_rejected PASSED [ 79%]
tests/test_adversarial.py::test_modified_dh_public_value_rejected PASSED [ 82%]
tests/test_adversarial.py::test_unexpected_peer_identity_rejected PASSED [ 86%]
tests/test_adversarial.py::test_malformed_transcript_rejected_before_hashing PASSED [ 89%]
tests/test_adversarial.py::test_dh_values_and_nonces_are_fresh_per_session PASSED [ 93%]
tests/test_adversarial.py::test_handshake_discards_dh_private_values PASSED [ 96%]
tests/test_limitations.py::test_ctr_counter_overlap_for_multi_block_records PASSED [100%]

===== 29 passed in 0.54s =====
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$
```

Security Note:

The attacker is in the middle of the gateway and node. It can read, modify, drop, reorder, replay, inject, and reflect records. They also know the protocol and message formats. It is important to note that the attacker does not have RSA private keys or session keys. Some non-goals include compromised endpoints, malicious but authenticated peer, traffic analysis, denial of service, and a secure distribution of RSA public keys if that are already trusted.

CTR requires unique IVs because it turns AES into a keystream and the ciphertext is the plaintext XORed with it. If the same key and IV are used again, the same keystream is used so $C1 \oplus C2 = P1 \oplus P2$. The stream is stopped and a known plaintext is revealed. Here, the IV is `session_id || sequence` so it is special per session and record, and each direction has its own key. This layout reuses stream blocks for messages over 16 bytes because the CTR counter runs into the next sequence number. For example, a 32-byte record at sequence 0 uses counter blocks 0 and 1, and the next record at sequence 1 starts at block 1, so one keystream block is used twice.

Verify-before-decrypt matters because you don't want to process data you haven't checked yet. It could be malicious. Modified ciphertext should never reach a decryption stage or tool executor. This avoids releasing plaintext derived from the attacker chosen input. This rejects forgeries and keeps the receiver's state from being changed by un-checked records.

Independent encryption and MAC keys are used because they are separate. If one key is compromised, the other key remains intact. A leaked encryption key doesn't let someone forge records. Also, using one key for two primitives risks unintended things, and security proofs assume independent keys. The KDF's distinct labels give separate keys for each purpose and direction which is also why reflected records fail.

In terms of construction with an AEAD mode such as AES-GCM, AEAD does encryption and authentication in one primitive with one key. So it authenticates headers through similar data. It brings about less errors since there is less to get wrong in comparison. Another valid method is encrypt-then-mac with independent keys when performed right. However, it relies more on the developer to verify first and cover the right fields. GCM also needs unique nonces and reusing one is really bad because it can leak an authentication key, not just plaintext relationships.

A valid protected record may still contain a dangerous authorized tool call because cryptography says nothing about whether the content is safe. It proves who sent a message, that it is unmodified, and that it is fresh. An authenticated gateway could be compromised, with something like prompt injection, or simply sends a harmful command. Protection is not enough in this spot as you need allow-lists, sandboxes, approval for risky commands, and logging. In the end, cryptography can't make an endpoint trustworthy. For example a correctly signed and sequenced WIPE command would pass every check and still be harmful.